

# 动态资源分配 DRA

动态资源分配在 1.32 版本中升级到 Beta，默认关闭，只有在启用 DynamicResourceAllocation 特性门控 和 [resource.k8s.io/v1beta1](https://kubernetes.io/v1beta1) API 组 时才启用。有关详细信息，参阅 `--feature-gates` 和 `--runtime-config kube-apiserver` 参数。kube-scheduler、kube-controller-manager 和 kubelet 也需要设置该特性门控。

之前的方式是使用 device-plugin，但是存在以下问题：

- 设备初始化：当启动一些使用 FPGA 的应用时，在启动应用之前，需要重新配置 FPGA 或者重新编程 FPGA。
- 设备清理：当应用运行完成时，需要清理这个应用在设备上的配置，数据等信息。例如，FPGA 需要重置。目前的 Device plugin 机制并没有包含清理操作的接口。
- 部分分配：某些场景下，允许一个 pod 中某个容器使用某个设备的部分资源，并且该 pod 的其他容器可以使用这个设备的剩余资源。例如：新一代 GPU 支持 MIG 模式，允许将一张 GPU 卡划分为更小的 GPU（称为 GPU 实例，MIG）。在一张 GPU 卡上划分 MIG 设备是一个动态过程，当有某个 pod 使用 MIG 时，应该先为其划分出一个 MIG 实例，当该 pod 运行完成后，需要销毁这个 MIG 实例。
- 可选分配：部署工作负载时，用户希望指定软（可选）设备要求，如果设备存在并且可分配，那么执行分配操作。如果设备不存在，那么将回退到没有设备的节点上运行。GPU 和加密引擎就是此类例子。如果一个节点有 GPU 设备，那么使用 GPU 设备运行；如果 GPU 没有，那么回退到使用 CPU 来运行同一任务。
- 带有拓扑性质的设备分配：某些设备分配时，需要考虑拓扑性质，比如 RDMA 和 GPU。
- 资源选择的精细程度有限：只能通过一些节点属性和标签来对节点上的资源类型进行相对粗略的选择，无法进行更精细的控制。
- 缺乏有效的调度反馈机制：设备插件和 kube-scheduler 之间缺乏有效的反馈回路。这意味着，即使 kube-scheduler 为一个待调度的 Pod 找到了看似合适的节点，设备插件也无法根据节点的就绪状态等关键因素进行进一步的“校验”，这可能导致 Pod 被错误地调度到尚未完全准备好接收新资源的“未就绪”节点上。

## DeviceClass [🔗](#)

DeviceClass 定义了设备的特性。它指定了设备的驱动程序和参数，为请求资源提供了一种结构化的方式。以下是 DeviceClass 的示例：

```
1  apiVersion: resource.k8s.io/v1beta1
2  kind: DeviceClass
3  metadata:
4    name: resource.example.com
5  spec:
6    selectors:
7  - cel:
8    expression: device.driver == "resource-driver.example.com"
```

## ResourceClaimTemplates [🔗](#)

ResourceClaimTemplate 是用于生成 ResourceClaim 对象的模板。当一个 pod 引用一个 ResourceClaimTemplate 时，为 pod spec 的 resourceClaims 部分的每个条目生成一个新的 ResourceClaim。以下是 ResourceClaimTemplate 的示例：

```
1  apiVersion: resource.k8s.io/v1beta1
2  kind: ResourceClaimTemplate
3  metadata:
4    name: large-black-cat-claim-template
5  spec:
6    spec:
7      devices:
8        requests:
9        - name: req-0
10     deviceClassName: resource.example.com
```

```

11     selectors:
12     - cel:
13         expression: |-
14             device.attributes["resource-driver.example.com"].color == "black" &&
15             device.attributes["resource-driver.example.com"].size == "large"

```

## ResourceSlice [🔗](#)

一个 ResourceSlice 表示节点上可用资源的一个切片。它用于管理和分配资源给 ResourceClaims。以下是 ResourceSlice 的示例：

```

1  apiVersion: resource.k8s.io/v1beta1
2  kind: ResourceSlice
3  metadata:
4    name: slice1
5  spec:
6    devices:
7    - basic:
8        attributes: # 设备固有属性
9        family:
10         string: Arc # 设备系列标识 (如 GPU 架构)
11        model:
12         string: A770 # 设备型号标识
13        capacity: # 设备资源容量
14        memory: 16288Mi # 显存容量 (16GiB)
15        millicores: 1k # 计算核心数 (1000 毫核=1 核)
16        name: 0000-03-00-0-0x56a0 # 设备全局唯一标识符 (PCI 地址+硬件 ID)
17        driver: device.com # 驱动名称 (需符合 DNS 域名规范)
18        nodeName: node1 # 设备所在节点名称
19    pool:
20        generation: 0 # 资源池版本号 (更新时递增)
21        name: pool1 # 资源池名称 (通常与节点同名)
22        resourceSliceCount: 1 # 池中资源切片总数

```

## ResourceClaim [🔗](#)

一个 ResourceClaim 是一个请求特定类型资源的对象，用于为 Pod 分配资源。ResourceClaim 与 Persistent Volume Claim (PVC) 类似，但用于设备资源。以下是部署示例 pod 后，生成的 ResourceClaim 的示例：

```

1  apiVersion: resource.k8s.io/v1beta1
2  kind: ResourceClaim
3  metadata:
4    annotations:
5      resource.kubernetes.io/pod-claim-name: cat-0
6    name: pod-with-cats-cat-0-bvrj
7    namespace: default
8  spec:
9    devices:
10    requests:
11    - allocationMode: ExactCount
12      count: 1
13      deviceClassName: resource.example.com
14      name: req-0
15      selectors:
16      - cel:
17          expression: |-
18              device.attributes["resource-driver.example.com"].color == "black" &&
19              device.attributes["resource-driver.example.com"].size == "large"

```

```
20 status: {}
```

## Pods 申请动态资源示例 [🔗](#)

```
1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: pod-with-cats
5  spec:
6    containers:
7    - name: container0
8      image: ubuntu:20.04
9      command: ["sleep", "9999"]
10     resources:
11       claims:
12       - name: cat-0
13     - name: container1
14       image: ubuntu:20.04
15       command: ["sleep", "9999"]
16       resources:
17         claims:
18         - name: cat-1
19     resourceClaims:
20     - name: cat-0
21       resourceClaimTemplateName: large-black-cat-claim-template
22     - name: cat-1
23       resourceClaimTemplateName: large-black-cat-claim-template
```

## 组件 [🔗](#)

### Resource Controller [🔗](#)

与 Kubernetes 调度器协调的集中式组件，以决定哪些节点可以为传入的资源请求提供服务。它处理 `ResourceClaim` 和 `ResourceSlice` 对象的创建和管理。

### Kubelet Plugin [🔗](#)

每个节点部署一个类似于 Device Plugin 的 Kubelet Plugin，运行在每个节点上的守护进程，它与硬件交互，执行资源的发现、分配、准备和取消准备。它向控制器报告可用资源，并确保这些资源被正确配置以供 pod 使用。开发一个 Kubelet Plugin 需要实现两个函数：

```
1 // NodeServer is the server API for Node service.
2 type NodeServer interface {
3
4     NodePrepareResource(context.Context, *NodePrepareResourceRequest) (*NodePrepareResourceResponse, error)
5
6     NodeUnprepareResource(context.Context, *NodeUnprepareResourceRequest) (*NodeUnprepareResourceResponse, error)
7 }
```

对这两个函数说明如下：

- `NodePrepareResource`：Pod 被调度到节点以后，kubelet 在创建该 Pod 前，会通过 GRPC 调用 Kubelet Plugin 的这个函数，Plugin 会在节点上准备相关资源，同时也提供 Device Plugin 类似的能力，将需要挂载到容器的设备路径，设置环境变量等信息返回给 Kubelet。
- `NodeUnprepareResource`：Pod 被删除时，该函数被触发，用于执行某些与 pod 相关的资源清理。

## minikube 环境 [🔗](#)

AlmaLinux 系统安装 minikube 环境：

```

1  sudo yum install -y yum-utils
2  sudo yum-config-manager --add-repo <https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo>
3  sudo yum install -y docker-ce docker-ce-cli containerd.io
4  sudo systemctl enable --now docker
5  docker --version
6  # docker 镜像加速
7  sudo tee /etc/docker/daemon.json <<-'EOF'
8  {
9    "registry-mirrors": ["<https://n766si7p.mirror.aliyuncs.com>"]
10 }
11 EOF
12 sudo systemctl daemon-reload
13 sudo systemctl restart docker
14 # 获取镜像
15 docker pull registry.cn-hangzhou.aliyuncs.com/google_containers/kicbase:v0.0.46
16 docker tag registry.cn-hangzhou.aliyuncs.com/google_containers/kicbase:v0.0.46 gcr.io/k8s-minikube/kicbase:v0.0.46
17 # 启动
18 minikube start --force --driver=docker --image-repository=registry.aliyuncs.com/google_containers \
19 --feature-gates=DynamicResourceAllocation=true \
20 --extra-config=apiserver.feature-gates="DynamicResourceAllocation=true" \
21 --extra-config=apiserver.runtime-config="resource.k8s.io/v1beta1" \
22 --extra-config=scheduler.feature-gates="DynamicResourceAllocation=true" \
23 --extra-config=controller-manager.feature-gates="DynamicResourceAllocation=true" \
24 --extra-config=kubelet.feature-gates="DynamicResourceAllocation=true"
25 # 查看安装后是否有 api
26 kubectl api-resources | grep resource.k8s.io/v1beta1

```

## 组件示例 demo dra-example-driver [🔗](#)

有两个示例 demo 如下：

- 🔗 [GitHub - kubernetes-sigs/dra-example-driver: Example DRA driver that developers can fork and modify to get them started writing their own.](#)
- 🔗 [GitHub - NVIDIA/k8s-dra-driver-gpu: Dynamic Resource Allocation \(DRA\) for NVIDIA GPUs in Kubernetes](#)

第一个代码 demo dra-example-driver 中，main 文件中主要代码：

```

1  // 定义驱动核心参数（需与 Kubernetes DRA 机制配合）
2  const (
3      DriverName = "gpu.example.com" // 驱动唯一标识符，需匹配 ResourceClass 中的 driverName
4
5      // kubelet 插件注册路径（遵循 Kubernetes 设备插件规范）
6      PluginRegistrationPath = "/var/lib/kubelet/plugins_registry/" + DriverName + ".sock"
7      DriverPluginPath       = "/var/lib/kubelet/plugins/" + DriverName // 插件工作目录
8      DriverPluginSocketPath = DriverPluginPath + "/plugin.sock"      // gRPC 通信 socket
9      DriverPluginCheckpointFile = "checkpoint.json"                  // 设备分配状态持久化文件
10 )
11
12 // CLI 参数配置（启动时通过命令行或环境变量注入）
13 func newApp() *cli.App {
14     flags := &Flags{
15         loggingConfig: flags.NewLoggingConfig(),
16     }
17     cliFlags := []cli.Flag{
18         &cli.StringFlag{
19             Name:     "node-name", // 必须指定节点名称（用于节点级资源管理）
20             Usage:    "The name of the node to be worked on.",
21             Required: true,        // 确保驱动实例与节点绑定
22             Destination: &flags.nodeName,

```

```

23     EnvVars: []string{"NODE_NAME"},
24 },
25 &cli.StringFlag{
26     Name:     "cdi-root",    // CDI 配置文件生成目录（实现设备注入）
27     Usage:    "Absolute path to the directory where CDI files will be generated.",
28     Value:    "/etc/cdi",    // 默认路径符合 CDI 规范
29     Destination: &flags.cdiRoot,
30     EnvVars:   []string{"CDI_ROOT"},
31 },
32 &cli.IntFlag{
33     Name:     "num-devices", // 模拟设备数量（测试用参数）
34     Usage:    "The number of devices to be generated.",
35     Value:    8,             // 默认生成 8 个虚拟设备
36     Destination: &flags.numDevices,
37     EnvVars:   []string{"NUM_DEVICES"},
38 },
39 }
40 //...
41 }
42
43 // 插件主入口（实现生命周期管理）
44 func StartPlugin(ctx context.Context, config *Config) error {
45     // 创建插件工作目录（需符合 kubelet 插件规范）
46     err := os.MkdirAll(DriverPluginPath, 0750)
47     if err != nil {
48         return err
49     }
50
51     // 检查 CDI 目录状态（用于生成容器设备注入配置）
52     info, err := os.Stat(config.flags.cdiRoot)
53     switch {
54     case err != nil && os.IsNotExist(err):
55         err := os.MkdirAll(config.flags.cdiRoot, 0750) // 初始化 CDI 目录
56         if err != nil {
57             return err
58         }
59     case err != nil:
60         return err
61     case !info.IsDir():
62         return fmt.Errorf("path for cdi file generation is not a directory: '%v'", err)
63     }
64
65     // 初始化驱动核心逻辑（设备发现、资源分配等）
66     driver, err := NewDriver(ctx, config)
67     if err != nil {
68         return err
69     }
70
71     // 信号处理：实现优雅关闭
72     sigc := make(chan os.Signal, 1)
73     signal.Notify(sigc, syscall.SIGHUP, syscall.SIGINT, syscall.SIGTERM, syscall.SIGQUIT)
74     <-sigc // 阻塞等待终止信号
75
76     // 驱动关闭逻辑（释放资源、清理 CDI 文件等）
77     err = driver.Shutdown(ctx)
78     if err != nil {
79         klog.FromContext(ctx).Error(err, "Unable to cleanly shutdown driver")
80     }

```

```

81
82     return nil
83 }

```

driver.go 文件中主要代码：

```

1 // 实现 DRA 插件核心接口（符合 k8s DRA v1beta1 规范）
2 var _ drapbv1.DRAPluginServer = &driver{}
3
4 type driver struct {
5     client coreclientset.Interface // Kubernetes API 客户端
6     plugin kubeletplugin.DRAPlugin // kubelet 插件实例
7     state *DeviceState // 设备状态管理器
8 }
9
10 // NewDriver 初始化驱动核心组件（设备状态管理 + kubelet 插件注册）
11 func NewDriver(ctx context.Context, config *Config) (*driver, error) {
12     driver := &driver{
13         client: config.coreclient,
14     }
15
16     // 初始化设备状态（包含可分配设备列表）
17     state, err := NewDeviceState(config) // 管理设备生命周期
18     if err != nil {
19         return nil, err
20     }
21     driver.state = state
22
23     // 启动 kubelet 插件（关键注册流程）
24     plugin, err := kubeletplugin.Start(
25         ctx,
26         []any{driver},
27         kubeletplugin.KubeClient(config.coreclient), // 绑定 Kubernetes 客户端
28         kubeletplugin.NodeName(config.flags.nodeName), // 节点绑定
29         kubeletplugin.DriverName(DriverName), // 驱动名称标识
30         kubeletplugin.RegistrarSocketPath(PluginRegistrationPath), // 注册 socket 路径
31         kubeletplugin.PluginSocketPath(DriverPluginSocketPath), // 插件通信 socket
32         kubeletplugin.KubeletPluginSocketPath(DriverPluginSocketPath))
33     if err != nil {
34         return nil, err
35     }
36     driver.plugin = plugin
37
38     // 发布可分配设备资源（ResourceSlice 对应）
39     var resources kubeletplugin.Resources
40     for _, device := range state.allocatable {
41         resources.Devices = append(resources.Devices, device)
42     }
43     // 发布 ResourceSlice 对象资源，如果该函数从未被调用，那么 kubelet 插件将不会管理任何 ResourceSlice 对象。
44     // 该函数是非阻塞的，返回后可能仍需要一段时间才能将所有信息实际写入 API 服务器。调用者在调用该函数后，不得修改传入的内容。
45     if err := plugin.PublishResources(ctx, resources); err != nil {
46         return nil, err
47     }
48
49     return driver, nil
50 }
51
52 // NodePrepareResources 处理资源准备请求（kubelet 调用）
53 func (d *driver) NodePrepareResources(ctx context.Context, req *drapbv1.NodePrepareResourcesRequest) (*drapbv1.NodePrepareResourcesResponse, error) {

```

```

54 klog.Infof("NodePrepareResource is called: number of claims: %d", len(req.Claims))
55 preparedResources := &drapbv1.NodePrepareResourcesResponse{Claims: map[string]*drapbv1.NodePrepareResourceResponse{}}
56
57 // 遍历处理每个资源声明（支持批量处理）
58 for _, claim := range req.Claims {
59     preparedResources.Claims[claim.UID] = d.nodePrepareResource(ctx, claim)
60 }
61
62 return preparedResources, nil
63 }
64
65 func (d *driver) nodePrepareResource(ctx context.Context, claim *drapbv1.Claim) *drapbv1.NodePrepareResourceResponse {
66     // 获取 ResourceClaim 对象
67     resourceClaim, err := d.client.ResourceV1beta1().ResourceClaims(claim.Namespace).Get(
68         ctx,
69         claim.Name,
70         metav1.GetOptions{})
71     if err != nil {
72         return &drapbv1.NodePrepareResourceResponse{
73             Error: fmt.Sprintf("failed to fetch ResourceClaim %s in namespace %s", claim.Name, claim.Namespace),
74         }
75     }
76
77     // 调用设备状态管理器分配设备（实际资源分配逻辑）
78     prepared, err := d.state.Prepare(resourceClaim)
79     if err != nil {
80         return &drapbv1.NodePrepareResourceResponse{
81             Error: fmt.Sprintf("error preparing devices for claim %v: %v", claim.UID, err),
82         }
83     }
84
85     klog.Infof("Returning newly prepared devices for claim '%v': %v", claim.UID, prepared)
86     return &drapbv1.NodePrepareResourceResponse{Devices: prepared}
87 }
88
89 // NodeUnprepareResources 处理资源释放请求（Pod 终止时调用）
90 func (d *driver) NodeUnprepareResources(ctx context.Context, req *drapbv1.NodeUnprepareResourcesRequest) (*drapbv1.NodeUnprepareResourcesResponse, error) {
91     klog.Infof("NodeUnPrepareResource is called: number of claims: %d", len(req.Claims))
92     unpreparedResources := &drapbv1.NodeUnprepareResourcesResponse{Claims: map[string]*drapbv1.NodeUnprepareResourceResponse{}}
93
94     // 批量处理资源释放
95     for _, claim := range req.Claims {
96         unpreparedResources.Claims[claim.UID] = d.nodeUnprepareResource(ctx, claim)
97     }
98
99     return unpreparedResources, nil
100 }
101
102 func (d *driver) nodeUnprepareResource(ctx context.Context, claim *drapbv1.Claim) *drapbv1.NodeUnprepareResourceResponse {
103     // 释放设备资源（更新设备状态）
104     if err := d.state.Unprepare(claim.UID); err != nil {
105         return &drapbv1.NodeUnprepareResourceResponse{
106             Error: fmt.Sprintf("error unpreparing devices for claim %v: %v", claim.UID, err),
107         }
108     }
109
110     return &drapbv1.NodeUnprepareResourceResponse{}
111 }

```

部署后，kubectl 上服务状态如下：

```
1 [root@instance-zztzxf ~]# kubectl -ndra-example-driver get ds
2 NAME                                DESIRED  CURRENT  READY  UP-TO-DATE  AVAILABLE  NODE SELECTOR  AGE
3 dra-example-driver-kubeletplugin 1        1        1        1          1          <none>         23h
4 [root@instance-zztzxf ~]# kubectl -ndra-example-driver get pods
5 NAME                                READY  STATUS  RESTARTS  AGE
6 dra-example-driver-kubeletplugin-gfqdr 1/1    Running  0          30m
7 # 启动后生成的 mock device
8 [root@instance-zztzxf ~]# kubectl get resourceslices
9 NAME                                NODE      DRIVER      POOL      AGE
10 minikube-gpu.example.com-bsdgr minikube  gpu.example.com minikube 23m
11 [root@instance-zztzxf ~]# kubectl get resourceslices minikube-gpu.example.com-bsdgr -oyaml
12 apiVersion: resource.k8s.io/v1beta1
13 kind: ResourceSlice
14 metadata:
15   generateName: minikube-gpu.example.com-
16   name: minikube-gpu.example.com-bsdgr
17 spec:
18   devices:
19   - basic:
20     attributes:
21     driverVersion:
22       version: 1.0.0
23     index:
24       int: 7
25     model:
26       string: LATEST-GPU-MODEL
27     uuid:
28       string: gpu-b6c22e51-162e-c4bd-c792-f4a023c8c6a4
29     capacity:
30     memory:
31       value: 80Gi
32     name: gpu-7
33   - basic:
34     attributes:
35     driverVersion:
36       version: 1.0.0
37     index:
38       int: 0
39     model:
40       string: LATEST-GPU-MODEL
41     uuid:
42       string: gpu-99c8e21f-63de-d9f3-e993-0b597f6cebd6
43     capacity:
44     memory:
45       value: 80Gi
46     name: gpu-0
47   - basic:
48     attributes:
49     driverVersion:
50       version: 1.0.0
51     index:
52       int: 1
53     model:
54       string: LATEST-GPU-MODEL
55     uuid:
56       string: gpu-5618b2a5-fede-1088-d397-cff74ffb8052
```



```
57     capacity:
58     memory:
59       value: 80Gi
60     name: gpu-1
61     # ...
62     driver: gpu.example.com
63     nodeName: minikube
64   pool:
65     generation: 1
66     name: minikube
67     resourceSliceCount: 1
```

这个 resourceslices 是上面代码函数 NewDeviceState 中调用 enumerateAllPossibleDevices mock 出来的。接着部署测试服务：

```
1  kubectl apply --filename=demo/gpu-test1.yaml
2  cat demo/gpu-test1.yaml
3  # Two pods, one container each
4  # Each container asking for 1 distinct GPU
5  ---
6  apiVersion: v1
7  kind: Namespace
8  metadata:
9    name: gpu-test1
10 ---
11 apiVersion: resource.k8s.io/v1beta1
12 kind: ResourceClaimTemplate
13 metadata:
14   namespace: gpu-test1
15   name: single-gpu
16 spec:
17   spec:
18     devices:
19       requests:
20       - name: gpu
21         deviceClassName: gpu.example.com
22
23 ---
24 apiVersion: v1
25 kind: Pod
26 metadata:
27   namespace: gpu-test1
28   name: pod0
29   labels:
30     app: pod
31 spec:
32   containers:
33   - name: ctr0
34     image: docker.yylt.gq/library/ubuntu:22.04
35     command: ["bash", "-c"]
36     args: ["export; trap 'exit 0' TERM; sleep 9999 & wait"]
37     resources:
38       claims:
39       - name: gpu
40     resourceClaims:
41     - name: gpu
42       resourceClaimTemplateName: single-gpu
43
44 ---
45 apiVersion: v1
```

```

46 kind: Pod
47 metadata:
48   namespace: gpu-test1
49   name: pod1
50   labels:
51     app: pod
52 spec:
53   containers:
54   - name: ctr0
55     image: docker.yylt.gq/library/ubuntu:22.04
56     command: ["bash", "-c"]
57     args: ["export; trap 'exit 0' TERM; sleep 9999 & wait"]
58   resources:
59     claims:
60     - name: gpu
61 resourceClaims:
62 - name: gpu
63   resourceClaimTemplateName: single-gpu

```

部署成功后，查看 pod 使用的 resourceClaim：

```

1  kubectl -ngpu-test1 get pods pod0 -oyaml | grep resourceClaimName
2    resourceClaimName: pod0-gpu-r9v5f
3  kubectl get resourceclaim pod0-gpu-r9v5f -n gpu-test1 -o yaml
4  apiVersion: resource.k8s.io/v1beta1
5  kind: ResourceClaim
6  metadata:
7    annotations:
8      resource.kubernetes.io/pod-claim-name: gpu
9    name: pod0-gpu-r9v5f
10   namespace: gpu-test1
11 spec:
12   devices:
13     requests:
14     - allocationMode: ExactCount
15       count: 1
16       deviceClassName: gpu.example.com
17       name: gpu
18 status:
19   allocation:
20     devices:
21       results:
22       - adminAccess: null
23         device: gpu-7
24         driver: gpu.example.com
25         pool: minikube
26         request: gpu
27   nodeSelector: # 限制使用该资源的 Pod 必须调度到指定节点
28     nodeSelectorTerms:
29     - matchFields:
30       - key: metadata.name
31         operator: In
32       values:
33       - minikube
34   reservedFor: # 表示此资源声明当前被保留给某个资源
35   - name: pod0
36   resource: pods
37   uid: b52c85ef-ae44-4658-b083-fe63e7e6745f

```

## 参考

- K8S 下一代设备管理机制：DRA(<https://developer.aliyun.com/article/1272914>)
-  [Dynamic Resource Allocation in Kubernetes - Blog](#)
-  [Delve into Dynamic Resource Allocation, devices, and drivers on Kubernetes](#)